UNIVERSIDAD DE GUAYAQUIL

FACULTAD DE INGENIERÍA INDUSTRIAL

ANTEPROYECTO DE TRABAJO DE TITULACIÓN

Datos Generales

CARRERA:	Sistemas de la información	
FECHA:	25/08/2026	
	APELLIDOS	**NOMBRES**
ESTUDIANTE 1:	Zambrano Mendoza	Jeremy Dario
ESTUDIANTE 2:	Chango Burgos	Michael Alexander
CORREO ELECTRÓNICO INSTITUCIONAL Y PERSONAL:	Jeremy.zambranome@ug.edu.ec / jdzmben10@hotmail.com michael.changob@ug.edu.ec / changoburgosma@gmail.com	
TELÉFONO:	0961851315	0991337480

Tema

TEMA	Librería y Api en Python para Operacionalizar la Metodología de un Gemelo Digital Multimodal Urbano-Industrial	
LÍNEA DE INVESTIGACIÓN	SOSTENIBILIDAD, DERECHOS Y TECNOLOGÍAS EN EL ORDENAMIENTO TERRITORIAL, AMBIENTE DE LA CONSTRUCCIÓN Y ARQUITECTURA.	
	DESARROLLO LOCAL Y EMPRENDIMIENTO SOCIO ECONÓMICO SOSTENIBLE.	
	CIENCIAS BÁSICAS, BIOCONOCIMIENTO Y DESARROLLO INDUSTRIAL.	**X**
	SALUD HUMANA, ANIMAL Y DEL AMBIENTE	
	TECNOLOGÍAS AGRÍCOLAS Y DE LA AGRICULTURA.	
	ENTORNOS VIRTUALES DE APRENDIZAJE Y TECNOLOGÍAS EDUCATIVAS	
COBERTURA	Nacional	
	Provincial	
	Local	**X**
	Otros	

1. PLANTEAMIENTO Y FORMULACIÓN DEL PROBLEMA. 4

1.1 Antecedentes. 4

1.2 Planteamiento del problema. 5

1.3 Causas. 5

1.4 Consecuencias. 5

1.5 Formulación del problema. 5

2. ALCANCE. 6

3. JUSTIFICACIÓN DE LA INVESTIGACIÓN. 6

3.1 Justificación teórica. 6

3.2 Justificación práctica. 7

3.3 Justificación metodológica. 7

4. OBJETIVO GENERAL. 7

5. OBJETIVOS ESPECÍFICOS. 7

6. HIPÓTESIS DE INVESTIGACIÓN. 7

7. MARCO TEÓRICO Y REFERENCIAL. 8

7.1 Gemelos digitales multimodales: concepto y evolución 8

7.2 Interoperabilidad y frameworks de código abierto 8

7.3 Arquitecturas composicionales: el caso OpenTwins 8

7.4 Arquitecturas de referencia, microservicios y estandarización de datos 8

7.5 Gemelos digitales aplicados a gestión de riesgo urbano y de desastres 8

7.6 Design Science Research como marco metodológico 9

8. MARCO METODOLÓGICO. 9

8.1 Tipo de investigación. 9

8.2 Enfoque y modalidad de investigación. 9

8.3 Fuentes, técnicas e instrumentos para la recolección de datos. 9

8.4 Población y muestra. 9

8.5 Análisis y tratamiento de la información. 10

8.6 Matriz de operacionalización de variables. 10

9. METODOLOGÍA DE DESARROLLO A EMPLEAR. 10

9.1 Justificación de la metodología. 10

9.2. Cuadro explicativo de la metodología Design Science Research. 11
9.3. Herramientas de planificación. 12
9.3.1 Fases de la metodología en el desarrollo de la librería y la API. 12
9.5. Prácticas aplicadas. 13
9.6. Historias de usuario / Requisitos. 13
10. CRONOGRAMA DE ACTIVIDADES. 14
11. PRESUPUESTO Y FINANCIAMIENTO. 14
11.1. Presupuesto de componente técnico. 14
11.2. Presupuesto del recurso operativo. 15
11.3. Financiamiento. 15
11.3.1. Presupuesto de componente técnico. 15
11.3.2. Presupuesto del recurso operativo. 15
12. BENEFICIOS ESPERADOS DEL PROYECTO. 15
13. DERECHOS DE PROPIEDAD INTELECTUAL. 16
14. REFERENCIAS BIBLIOGRÁFICAS. 16
ANEXO — Ficha técnica del producto. 19
RÚBRICA DE EVALUACIÓN. 20
ANEXO A — Rúbrica de Evaluación para Proyectos Individuales de Integración Curricular. 21
ANEXO C — Rúbrica de Evaluación para Proyectos Grupales de Integración Curricular. 22
FIRMAS. 23

1. PLANTEAMIENTO Y FORMULACIÓN DEL PROBLEMA.

1.1 Antecedentes.

En los últimos años, hablar de gemelos digitales era hablar casi exclusivamente de fábricas y de obra civil. Eso cambió: el concepto se movió hacia entornos urbanos e industriales mucho más complejos, donde ahora conviven datos geoespaciales, redes de infraestructura, modelos predictivos e interfaces de simulación, todo al mismo tiempo. La revisión sobre marcos de gemelos digitales de código abierto publicada en Software: Practice and Experience (2024) lo confirma: aunque cada vez aparecen más frameworks, la mayoría se construye por separado, sin una capa común que conecte módulos heterogéneos de datos, análisis e inteligencia artificial. OpenTwins (Robles, Martín y Díaz, 2023) va en esa dirección: propone una arquitectura composicional de código abierto pensada justamente para resolver esa fragmentación, e incorpora después modelos FMI y de aprendizaje automático dentro de la misma plataforma (Software: Practice and Experience, 2024). Y no es el único caso: makeTwin (2024) y la plataforma de gemelos digitales para plantas de tratamiento de agua basada en microservicios (Sensors, 2024) apuntan en la misma dirección, arquitecturas de referencia y servicios desacoplados, como respuesta a un mismo problema, la falta de interoperabilidad.

Asimismo, dentro de ese panorama general, el subcampo que más le interesa a este proyecto son los gemelos digitales aplicados a gestión de riesgo urbano y de desastres, está creciendo rápido, pero de forma igual de dispersa. La revisión sobre gemelos digitales de gestión de riesgo posdesastre, publicada en International Journal of Disaster Risk Reduction (Lagap y Ghaffarian, 2024), advierte que la mayoría de las aplicaciones existentes siguen dependiendo de recolección de datos automatizada que no siempre es viable en un escenario real de emergencia, lo que limita su implementación práctica más allá del laboratorio. Ariyachandra y Wedawatta (2023) llegan a una conclusión parecida al revisar gemelos digitales de ciudades inteligentes orientados a gestión de riesgo de desastres: el concepto avanza más rápido en la literatura que en implementaciones reales, y buena parte de esa brecha se explica por la falta de arquitecturas técnicas reutilizables que otros equipos puedan adoptar sin partir de cero. Weil, Bibri, Longchamp, Golay y Alahi (2023) documentan un obstáculo adicional, específico de los gemelos digitales urbanos: la integración de

fuentes de datos heterogéneas como sensores IoT, catastros, redes de infraestructura sigue siendo, en la práctica, uno de los cuellos de botella técnicos más persistentes del campo.

Macatulad y Biljecki (2024), retomando el balance intermedio del Marco de Sendai, señalan que buena parte del potencial de los gemelos digitales urbanos para la gestión de riesgo de desastres todavía está sin explotar, precisamente por la ausencia de plataformas técnicas que integren de forma sistemática los distintos componentes del análisis de riesgo.

1.2 Planteamiento del problema.

Los temas de anteproyectos del FCI No 46: Gemelo Digital Multimodal que se proponen en el presente ciclo de titulación (TI2 2026 - 2027) por parte de compañeros de la misma Facultad de Ingeniería Industrial (identificadas internamente como SI-01, SI-02, IND-SIS-01, URB-01, URB-02, URB-03, URB-04 y URB-05) persiguen la misma idea de fondo: modelar el riesgo urbano-industrial de forma integral. Pero se están construyendo como piezas sueltas, sin una estructura de software común que defina cómo deben conectarse los datos, los indicadores, los grafos, los modelos de IA, la simulación y la visualización. El problema se vuelve crítico en el momento del ensamblaje. Debido a que el Producto Mínimo Viable (MVP del FCI) exige que estos ocho subproductos funcionen como un sistema único, y es justo ahí donde las decisiones tomadas de forma aislada por cada equipo empiezan a chocar entre sí: un módulo espera coordenadas en un formato que otro no produce, un endpoint asume una estructura de datos que otro subproducto no valida, una convención de nombres no coincide con la que usa el módulo vecino. Ninguno de estos desajustes es un error técnico de alguna tesis en particular; es, simplemente, lo que pasa cuando ocho equipos resuelven al mismo tipo de decisión sin coordinarse.

Además, ninguno de los ocho equipos tiene, dentro de su propio alcance de tesis, la responsabilidad ni el tiempo de resolver esta capa de integración. Cada uno responde por su componente , captura de datos, indicadores, grafos, IA, simulación o visualización , y resolver cómo se conectan todos entre sí queda, por defecto, sin dueño. Esa ausencia de responsable técnico específico es, en el fondo, la raíz del problema: no hay quien defina los contratos de datos, los formatos comunes ni las convenciones de instalación que le permitirían al conjunto funcionar como un solo gemelo digital, y no como ocho productos que casualmente comparten un mismo tema de tesis.

Asimismo, la problemática, no es que falten componentes en dichos entregables, sino que falta un mecanismo reusable que los articule: una librería instalable y una API documentada que funcionen como columna vertebral técnica del gemelo digital, sin reemplazar el trabajo de cada tesis, sino dándole un lugar común donde conectarse.

1.3 Causas.

- Los subproductos de tesis (entregables del FCI No 46) se desarrollarán en paralelo, sin una especificación técnica compartida de interfaces, formatos de datos ni contratos de integración.
- No existe un paquete o repositorio base que centralice modelos de datos, endpoints y convenciones reutilizables entre los distintos módulos del gemelo digital.
- Tampoco hay prácticas mínimas de ingeniería de software como pruebas automatizadas, documentación técnica, control de versiones coordinadas y compartidas entre los equipos.

1.4 Consecuencias.

- Los subproductos podrían quedar como prototipos aislados, imposibles de ensamblar en un producto integrador dentro del plazo del FCI.
- Esfuerzo duplicado: Cada equipo terminaría resolviendo por su cuenta problemas de integración que en realidad corresponden a una capa común.
- Sería difícil demostrar, de forma reproducible, que el gemelo digital multimodal funciona como un sistema unificado y no como un conjunto de piezas sueltas.

1.5 Formulación del problema.

¿Cómo implementar una librería/API reusable que operacionalice la metodología del gemelo digital multimodal para riesgo urbano-industrial?

2. ALCANCE.

El proyecto consiste en diseñar e implementar georisk_twin, una librería en Python que se distribuye como paquete instalable y que, al mismo tiempo, se expone como API mediante FastAPI. Integra los módulos de datos, indicadores, grafos, inteligencia artificial, simulación y visualización que están desarrollando las distintas tesis del gemelo digital multimodal. En concreto, el alcance incluye:

- La definición de la estructura del paquete Python (empaquetado, dependencias, instalación).
- La construcción de endpoints API documentados que expongan las funcionalidades de cada módulo integrado.
- La elaboración de documentación técnica, ejemplos de uso reproducibles y una plantilla de contribución para que los equipos de las demás tesis puedan conectar sus subproductos.
- Pruebas unitarias mínimas que garanticen la estabilidad de las interfaces expuestas.
- Una demostración técnica de integración horizontal con al menos los módulos de datos, indicadores y un módulo analítico.

El proyecto NO incluye:

- El desarrollo completo de los módulos de datos, indicadores, grafos, IA, simulación o visualización en sí mismos, los cuales corresponden a las tesis SI-01, SI-02, IND-SIS-01 y URB-01 a URB-05.
- georisk_twin no reemplaza ningún subproducto: provee la estructura común sobre la cual estos se conectan.
- El despliegue en producción del gemelo digital completo a nivel institucional o municipal.

3. JUSTIFICACIÓN DE LA INVESTIGACIÓN.

3.1 Justificación teórica.

La literatura reciente sobre gemelos digitales de código abierto coincide en un punto: la interoperabilidad entre módulos sigue siendo uno de los principales obstáculos para su adopción práctica (Software: Practice and Experience, 2024; Robles, Martín y Díaz, 2023). A su vez, OpenTwins y makeTwin (2024) ya demostraron algo importante: definir una arquitectura de referencia común, en vez de dejar que cada componente crezca por su cuenta, reduce muchísimo la complejidad de integrarlo todo después. Este proyecto se ubica en esa misma línea y aporta evidencia aplicada sobre cómo una arquitectura ligera basada en Python, FastAPI y Pydantic puede funcionar como capa de integración para un gemelo digital multimodal enfocado específicamente en riesgo urbano-industrial, un dominio que todavía se ha explorado poco frente a las aplicaciones industriales o de infraestructura hídrica, como se pudo evidenciar en (Sensors, 2024).

3.2 Justificación práctica.

Desde el lado práctico, el desarrollo de esta librería, responde a una necesidad muy concreta que comparten ocho tesis de titulación que dependen entre sí. Tener una estructura común evita que cada equipo reinvente sus propios mecanismos de conexión, bajo el riesgo de incompatibilidades en la etapa final de integración del FCI y deja abierta la posibilidad de que futuros estudiantes o instituciones reutilicen el paquete en nuevos casos de riesgo urbano o industrial. En el fondo, entregar un paquete instalable, con API documentada y demo reproducible, convierte un problema de coordinación entre equipos en algo mucho más tangible: un activo técnico verificable.

3.3 Justificación metodológica.

Se optó por Design Science Research porque lo que realmente importa aquí es diseñar y evaluar un artefacto de software, la librería y la API que resuelva un problema práctico de integración, no comprobar una hipótesis explicativa. Esta metodología se complementa con prácticas de ingeniería de software e integración continua básica, lo que permite ir ajustando el diseño del paquete a medida que los módulos de las demás tesis se van entregando, y verificar en cada ciclo si la arquitectura propuesta realmente logra conectar los subproductos sin imponerles restricciones excesivas.

4. OBJETIVO GENERAL.

Implementar una librería/API reusable que integre los módulos de datos, indicadores, grafos, IA, simulación y visualización del gemelo digital multimodal para riesgo urbano-industrial.

5. OBJETIVOS ESPECÍFICOS.

- Diseñar la estructura base del paquete Python georisk_twin, incluyendo modelos de datos con Pydantic, servicios principales y el proceso de instalación.
- Desarrollar y documentar los endpoints de la API mediante FastAPI, garantizando su cobertura mediante pruebas unitarias mínimas.
- Integrar horizontalmente al menos los subproductos de datos, indicadores y un módulo analítico provenientes de las tesis SI-01, SI-02, IND-SIS-01 y URB-01 a URB-05.
- Evaluar el desempeño de la integración en términos de facilidad de instalación, tiempo de respuesta de los endpoints y reproducibilidad de la demostración técnica.

6. HIPÓTESIS DE INVESTIGACIÓN.

No aplica. Al tratarse de un proyecto de desarrollo tecnológico orientado a la construcción de un artefacto de software (librería y API), enmarcado en Design Science Research, no corresponde formular una hipótesis explicativa, sino criterios de aceptación y métricas de evaluación del artefacto, los cuales se detallan en las secciones 8 y 9.

7. MARCO TEÓRICO Y REFERENCIAL.

7.1 Gemelos digitales multimodales: concepto y evolución

De acuerdo con (Jeremiah et al., 2024), "un gemelo digital multimodal es la copia virtual de algo físico"; en este caso, se trata de un entorno urbano-industrial que junta datos de distinto tipo (geoespaciales, de redes, predictivos, de simulación) para mostrar casi en tiempo real cómo se comporta ese sistema. Asimismo, esta idea nació en la manufactura, pero ya no se queda ahí ni de cerca: se ha ido extendiendo hacia dominios mucho más heterogéneos, empujada en buena parte por los avances en inteligencia artificial, big data y computación en la nube . Los mismos autores son claros en algo: "La seguridad de los datos y la interoperabilidad entre módulos siguen sin resolverse del todo". Y ese segundo punto es justo el que le importa al georisk_twin, porque aquí el sistema representado no es una sola máquina, sino un entorno urbano-industrial completo , lo que multiplica, y bastante, las fuentes de datos que hay que hacer encajar entre sí.

7.2 Interoperabilidad y frameworks de código abierto

Gil Arboleda et al. (2024) revisaron catorce frameworks de gemelos digitales de código abierto y los compararon en diez dimensiones distintas. La conclusión se repite una y otra vez en la literatura: casi todas las plataformas nacen pegadas a un dominio específico, así que usarlas en otro contexto obliga a rehacer los modelos de datos y las interfaces de integración desde cero. Ese es, precisamente, el hueco que el georisk_twin quiere llenar para el riesgo urbano-industrial: nadie ha construido todavía algo pensado para este dominio en particular.

7.3 Arquitecturas composicionales: el caso OpenTwins

OpenTwins es quizás la referencia más cercana a lo que este proyecto intenta hacer. Robles et al. (2023) lo diseñaron como un framework de código abierto con arquitectura composicional: en vez de construir un bloque monolítico, ensamblas módulos independientes de machine learning, IoT y visualización 3D, bajo contratos de datos claros. Un año después, Infante et al. (2024) le sumaron a esa misma plataforma modelos de simulación FMI (Interfaz de Maqueta Funcional) y de aprendizaje automático, sin tocar el núcleo de la arquitectura original. Ese es exactamente el principio que georisk_twin quiere replicar, aunque a una escala mucho más pequeña.

7.4 Arquitecturas de referencia, microservicios y estandarización de datos

Hay más de un camino para llegar a algo parecido. Tao et al. (2024) propusieron makeTwin, una arquitectura de referencia que separa con claridad los datos, la lógica de negocio y la exposición de servicios; la probaron en una planta textil real. En un terreno completamente distinto, Rodríguez-Alonso et al. (2024) mostraron cómo

una arquitectura de microservicios pensada para computación en el borde permite meter módulos nuevos en una planta de tratamiento de agua sin romper lo que ya funcionaba. Pero nada de esto sirve de mucho si antes no se ordenan los datos: Conde et al. (2023) insisten en que primero hay que estandarizarlos, y proponen el ecosistema FIWARE justamente para eso, antes de intentar integrar cualquier cosa de forma horizontal.

7.5 Gemelos digitales aplicados a gestión de riesgo urbano y de desastres

Dentro de todo este panorama, hay un subcampo que crece rápido, pero de forma bastante desordenada: los gemelos digitales aplicados a riesgo urbano y desastres. Lagap y Ghaffarian (2024) señalan un problema práctico que casi nadie resuelve: la mayoría de estas aplicaciones dependen de recolección de datos automatizada que, en una emergencia real, simplemente no siempre está disponible. Eso limita mucho su uso más allá del laboratorio. Ariyachandra y Wedawatta (2023) llegan a algo parecido por otro camino: el concepto avanza más rápido en los papers que en la vida real, y buena parte de esa brecha se explica porque faltan arquitecturas técnicas que la gente pueda simplemente reutilizar. Weil et al. (2023), por su parte, ponen el dedo en otro problema que se repite constantemente en los gemelos digitales urbanos: integrar fuentes de datos tan distintas entre sí como sensores IoT, catastros, redes de infraestructura , sigue siendo uno de los cuellos de botella más difíciles de resolver.

7.6 Design Science Research como marco metodológico,

Hevner et al. (2024) plantean algo que parece obvio, pero que casi nadie hace bien: la credibilidad de una investigación basada en diseño no depende solo del resultado final, sino de qué tan transparente sea todo el proceso cómo se diseñó, cómo se evaluó, cómo quedó documentado. Para proyectos más pequeños, como este, con apenas dos personas en el equipo, Möller et al. (2022) proponen algo más práctico: tratar cada iteración del artefacto como una unidad de diseño que se puede evaluar por separado, en vez de esperar hasta el final para validar todo de golpe.

7.7 Kanban como metodología de desarrollo.

Kanban es el método de gestión visual del flujo de trabajo adoptado para organizar el desarrollo diario de georisk_twin. Consiste en un tablero dividido por estados como pendiente, en progreso, en revisión y terminado, que limitan la cantidad de tareas que pueden estar en curso al mismo tiempo, evitando que el equipo disperse su esfuerzo en varios frentes sin cerrar ninguno. El trabajo se organiza como un flujo continuo: una tarea nueva se incorpora en cuanto hay capacidad disponible para atenderla, y no en función de un periodo de tiempo fijo previamente cerrado. Un estudio reciente sobre la aplicación de Kanban en proyectos de software documenta que sus límites de trabajo en curso generan beneficios medibles de eficiencia, particularmente en contextos donde el equipo debe coordinarse con partes externas al proyecto (Ozkan, 2022).

7.8 API First Design como enfoque de desarrollo

API First Design es un enfoque de desarrollo de software en el que la interfaz de programación se diseña y se acuerda antes de implementar la lógica interna del sistema, invirtiendo el orden tradicional donde la API surge como subproducto del código ya escrito. Beaulieu et al. (2022), en una revisión que compara el estado del enfoque tanto en la academia como en la industria, concluyen que este orden de trabajo produce APIs mejor definidas, reduce el acoplamiento entre el proveedor de la API y quienes la consumen, y permite que distintos equipos avancen en paralelo sobre un mismo contrato ya acordado, en lugar de esperar a que la implementación esté terminada para empezar a integrarse. Para georisk_twin, este principio es central: al definir primero los contratos de datos en Pydantic y la forma de los endpoints, los equipos de las demás tesis pueden empezar a desarrollar contra esa especificación sin necesidad de que el paquete esté completamente implementado, evitando así que la integración quede atrapada al final del proyecto.

7.9 OpenAPI como estándar de documentación e interoperabilidad

OpenAPI es la especificación de referencia para describir APIs web de forma legible tanto por personas como por máquinas, permitiendo generar documentación, código cliente y pruebas de forma automática a partir de una misma definición formal. Casas et al. (2021), en un mapeo sistemático de la literatura sobre esta especificación, identifican que su uso mejora consistentemente actividades de desarrollo como la generación automática de documentación, la validación de contratos entre servicios y la generación de código cliente en distintos lenguajes, consolidándola como el estándar de facto para describir APIs RESTful. En georisk_twin, FastAPI genera esta especificación de forma automática a partir de las rutas y los modelos Pydantic ya definidos, lo que asegura que la documentación de la API nunca quede desactualizada respecto al código: nace directamente de él.

8. MARCO METODOLÓGICO.

8.1 Tipo de investigación.

La investigación es de tipo aplicada , debido a que está orientada a desarrollar un artefacto tecnológico, librería y API que resuelva un problema de integración real: coordinar ocho tesis de titulación que forman parte del mismo gemelo digital multimodal. No se busca generar conocimiento teórico nuevo sobre el riesgo urbano-industrial como fenómeno, sino producir y evaluar una solución de ingeniería de software que otros puedan reutilizar.

8.2 Enfoque y modalidad de investigación.

El enfoque combina dos cosas: es cualitativo en el diseño arquitectónico del artefacto, decisiones de estructura, contratos de integración, documentación y cuantitativo en su evaluación, tiempo de respuesta de los endpoints, cobertura de pruebas, número de módulos integrados. En conjunto, es un enfoque mixto.

La modalidad es de campo y experimental de laboratorio: el artefacto se construye y se prueba de forma iterativa, integrando datasets reales de los subproductos urbanos e industriales.

8.3 Fuentes, técnicas e instrumentos para la recolección de datos.

Tipo de fuente	Fuente	Técnica	Instrumento
Primaria	Subproductos de las tesis SI-01, SI-02, IND-SIS-01, URB-01 a URB-05	Pruebas de integración horizontal	Suite de pruebas unitarias (pytest)
Primaria	Endpoints de la API georisk_twin	Medición de tiempo de respuesta	Herramientas de benchmarking sobre FastAPI
Secundaria	Literatura académica sobre frameworks de gemelos digitales de código abierto	Revisión documental	Fichas bibliográficas en formato APA

8.4 Población y muestra.

La población de este estudio son los ocho subproductos de tesis que deben integrarse al gemelo digital: SI-01, SI-02, IND-SIS-01, URB-01, URB-02, URB-03, URB-04 y URB-05. Como el número de subproductos es reducido y se conoce de antemano, no tiene sentido aplicar un muestreo estadístico. La evaluación se hará sobre

la totalidad de los módulos disponibles en cada iteración, priorizando en la primera versión (v0.5) al menos los módulos de datos, indicadores y un módulo analítico.

8.5 Análisis y tratamiento de la información.

Los resultados de las pruebas de integración y los tiempos de respuesta de los endpoints se irán registrando de forma sistemática en cada iteración del desarrollo, lo que permitirá comparar el desempeño del paquete a medida que se suman módulos nuevos. La cobertura de pruebas unitarias se calculará con las herramientas estándar de pytest, y los resultados de instalación y reproducibilidad quedarán documentados dentro de la guía técnica entregable, para que los equipos tutores puedan verificarlos de forma independiente.

8.6 Matriz de operacionalización de variables.

Variable	Dimensión	Indicador	Técnica	Instrumento
Integración de módulos	Cobertura funcional	Número de módulos integrados sobre el total de subproductos	Pruebas de integración horizontal	pytest
Calidad del software	Confiabilidad	Cobertura mínima de pruebas unitarias (%)	Análisis de cobertura	pytest-cov
Desempeño de la API	Eficiencia	Tiempo de respuesta de los endpoints	Benchmarking	Herramientas de medición sobre FastAPI
Usabilidad técnica	Facilidad de adopción	Facilidad de instalación del paquete	Prueba de instalación en entorno limpio	pip / entorno virtual
Reproducibilidad	Consistencia	Éxito de la demo técnica en distintos entornos	Ejecución de demo reproducible	Script de demostración documentado
Documentación	Completitud	Existencia de documentación Markdown/OpenAPI actualizada	Revisión documental	Repositorio en GitHub

9. METODOLOGÍA DE DESARROLLO A EMPLEAR.

9.1 Justificación de la metodología.

Se adoptó Design Science Research como marco general de investigación, junto con prácticas de ingeniería de software e integración continua básica. Tiene sentido para este proyecto por dos razones: el equipo es pequeño, de dos integrantes, y el trabajo depende de entregas externas de otros equipos de tesis. Design Science Research permite tratar cada versión del paquete (v0.5, por ejemplo) como una iteración de diseño que se puede evaluar; la integración continua básica, por su parte, garantiza que cada módulo incorporado se valide automáticamente antes de sumarse al resto del sistema, lo que reduce el riesgo de romper algo que ya funcionaba.

Asimismo, para el desarrollo de la librería y la API se utilizará la metodología KANBAN, debido a que responde directamente a cómo se manejará el proyecto. El equipo lo conforman dos personas, lo que exige una forma de organizar el trabajo simple y sin ceremonias que consuman tiempo de desarrollo. El flujo continuo de tarjetas, sin bloques cerrados de tiempo, permite tomar y priorizar tareas conforme van estando disponibles, algo especialmente útil cuando el desarrollo depende de entregas externas de ocho equipos de tesis distintos (SI-01, SI-02, IND-SIS-01, URB-01 a URB-05), cuyo cronograma no está bajo el control directo de este proyecto.

Se adoptará API First Design como enfoque de desarrollo porque el problema central de este proyecto no es construir una aplicación con una API incidental, sino construir precisamente la interfaz de integración entre ocho subproductos de tesis independientes. En ese contexto, diseñar la API antes de escribir la lógica interna del paquete no es una preferencia estilística: es la única forma de que los equipos de las demás tesis sepan, desde temprano, contra qué estructura de datos y qué endpoints deben desarrollar sus propios módulos, sin tener que esperar a que georisk_twin esté terminado para empezar a integrarse.

OpenAPI se adopta como estándar de implementación porque es la contraparte técnica natural de ese enfoque: una vez que la API se diseña primero, OpenAPI permite expresar ese diseño en una especificación formal, legible tanto por personas como por herramientas automatizadas. FastAPI genera esta especificación automáticamente a partir de las rutas y los modelos Pydantic ya definidos, así que la documentación nunca queda desactualizada respecto al código; nace directamente de él, no se escribe aparte ni después. Para un proyecto donde el entregable formal incluye explícitamente una "API documentada" como criterio de aceptación, esto no es un beneficio adicional: es el mecanismo que hace posible cumplir ese criterio sin duplicar trabajo entre desarrollar la API y documentarla por separado.

9.2. Cuadro explicativo de la metodología Design Science Research.

En el siguiente cuadro, se relacionarán las fases que se siguen en la metodología Design Science Research con el proceso a seguir en el desarrollo de la librería y la API para operacionalizar la metodología de un gemelo digital multimodal.

Fase de metodología DSR.	**Descripción alineada al desarrollo de la investigación.**
Identificación del problema.	En esta fase se realizó un diagnóstico y se comprobó que cada uno de estos componentes se desarrolla actualmente como un subproducto de tesis independiente, sin una estructura común que permita conectarlos entre sí.
Definición de objetivos de solución.	En esta fase se identificó el objetivo que tendrá esta investigación y el artefacto que se presentará como solución para la problemática, el cual será un paquete instalable y una API documentada, capaz de demostrarse mediante una demo reproducible, y con la conexión funcional de al menos los módulos de datos, indicadores y un módulo analítico como umbral mínimo para la versión v0.5.
Diseño y desarrollo.	En esta fase se realizará el desarrollo del paquete georisk_twin como estructura común de integración, sin sustituir ninguno de los subproductos de tesis existentes. Se desarrolla el empaquetado instalable en Python, los endpoints de la API mediante FastAPI y los contratos de datos en Pydantic que definen cómo debe conectarse cada uno de los seis módulos del gemelo digital, que son los de datos, indicadores, grafos, IA, simulación y visualización, sin imponerles restricciones sobre su implementación interna.
Demostración.	En esta fase se presentará el artefacto propuesto mediante una demo de integración que forma parte del entregable definido para esta fase del proyecto. Dicha demo debe mostrar, de manera reproducible, la conexión efectiva de al menos los módulos de datos, indicadores y un módulo analítico sobre el paquete georisk_twin v0.5, evidenciando que la estructura común de integración funciona sin necesidad de modificar los subproductos de tesis que la alimentan. La demostración se

	documenta de forma que cualquier persona con acceso al repositorio pueda instalar el paquete, levantar la API y reproducir la integración descrita.
Evaluación.	En esta fase se verificará el cumplimiento de los cuatro criterios de aceptación anteriormente. establecidos, que son los siguientes: que el paquete sea efectivamente instalable, que la API cuente con documentación completa, que la demo resulte reproducible por terceros y que la conexión mínima de datos, indicadores y un módulo analítico esté funcionando correctamente. A su vez, se ejecutará la suite de pruebas con pytest, se medirá la cobertura de código y se automatizará esta validación mediante GitHub Actions en cada actualización del repositorio. Asimismo, si alguno de los cuatro criterios no se cumple, el artefacto no se considera apto para cerrar la iteración v0.5, y el proceso retorna a la fase de diseño y desarrollo.
Comunicación.	En esta última fase, se entregará el paquete georisk_twin v0.5, acompañado de los tres elementos adicionales que completan el entregable definido para esta etapa: La API documentada, la demo de integración y la guía técnica. Dicha guía técnica describirá el proceso de instalación, la estructura de los contratos de integración y los pasos necesarios para que un nuevo subproducto de tesis se conecte a la estructura común sin alterarla.

9.3. Herramientas de planificación.

Para organizar el trabajo, se usará un tablero Kanban en Trello, separando las tareas de backend/API de las de integración y calidad. Esto permite ver el avance de cada responsabilidad por separado y coordinar los puntos donde el proyecto depende de otros equipos de tesis.

9.3.1 Fases de la metodología en el desarrollo de la librería y la API.

Fase 1: Visualización del flujo de trabajo.

Se construye el tablero en Trello, y se separan las columnas de la siguiente manera: Backlog, Por hacer, En progreso, En revisión, Bloqueado y Hecho, separando desde el inicio las tareas de backend/API, las cuales son:

diseño de la estructura del paquete, diseño de modelo Pydantic, desarrollo de endpoints principales de la API (FastAPI), de las tareas de integración y calidad, como lo son: integración del módulo de datos y de indicadores, integración de un módulo analítico (IA), pruebas unitarias y ajuste de cobertura mínima, desarrollo de documentación técnica, ejemplos de uso y plantilla de contribución. A su vez, con esta separación se permitirá identificar las actividades correspondientes al desarrollo del paquete y las actividades que se realizarán en la integración y validación del mismo.

Fase 2: Priorización de tarjetas por iteración.

Las tarjetas se priorizarán en función de las iteraciones cortas alineadas con los hitos de entrega de las demás tesis. En la primera iteración, se priorizarán las tarjetas que corresponden al backend/API, como lo son : diseño de la estructura del paquete, diseño de modelo Pydantic, desarrollo de endpoints principales de la API (FastAPI).

En la segunda iteración, se priorizarán las tareas de integración de los entregables de las demás tesis correspondientes al módulo de datos, de indicadores y del módulo analítico (IA).

Fase 3: Desarrollo con límite de trabajo en curso (WIP).

Cada integrante tomará tarjetas de "Por hacer", verificando el tablero visual en Trello, el número de actividades que ya se han terminado y avanzando a la siguiente según el orden establecido previamente por cada uno en el mismo, evitando así que se genere una inconsistencia en el desarrollo de cada una de ellas, y que exista la duda de si las tareas se han realizado correctamente cuando ya se realice la presentación del entregable final del proyecto.

Fase 4: Gestión de bloqueos con mocks y contratos de datos.

Cuando una tarjeta depende de un subproducto de tesis que aún no ha entregado su módulo, se mueve a la columna "Bloqueado" y se resuelve, siguiendo la práctica ya establecida en el documento, mediante mocks y contratos de datos definidos en Pydantic. Esto permite seguir desarrollando y probando los endpoints de FastAPI contra datos simulados que respetan la misma estructura que se espera del subproducto real, sin detener el avance del paquete.

Fase 5: Revisión de código.

Antes de fusionar cualquier cambio a la rama principal, la tarjeta pasa a "En revisión" y el otro integrante evalúa el código, ya sea un endpoint nuevo en FastAPI, un modelo de validación en Pydantic o una función de análisis con scikit-learn, antes de aprobar el pull request en GitHub.

Fase 6: Validación automática y cierre.

Al fusionarse el cambio, GitHub Actions ejecuta automáticamente la suite de pruebas con pytest y calcula la cobertura de código. Solo si estas pruebas pasan, la tarjeta se mueve a "Hecho". Esta fase es la que convierte cada tarjeta cerrada en una pieza verificada del paquete georisk_twin, y no solo en código que "parece funcionar".

9.5. Prácticas aplicadas.

- Iteraciones cortas alineadas con los hitos de entrega de los subproductos de las demás tesis.
- Integración continua básica mediante GitHub Actions, ejecutando pruebas unitarias en cada cambio.
- Revisión de código entre los dos integrantes antes de fusionar cambios a la rama principal.
- Uso de mocks y contratos de datos para avanzar en el desarrollo del paquete sin depender por completo de la disponibilidad simultánea de todos los módulos externos.

9.6. Historias de usuario / Requisitos.

ID	Historia de usuario	Prioridad	Iteración
HU-01	Como desarrollador del equipo, quiero instalar el paquete georisk_twin mediante pip para poder integrarlo rápidamente en mi entorno de trabajo.	Alta	1
HU-02	Como estudiante responsable de un subproducto de tesis, quiero contar con modelos Pydantic estandarizados para poder enviar mis datos en un formato compatible con la API.	Alta	1
HU-03	Como equipo tutor, quiero consultar la documentación OpenAPI de los endpoints para poder verificar qué funcionalidades expone el paquete.	Media	2
HU-04	Como desarrollador del equipo, quiero ejecutar pruebas unitarias automatizadas para poder detectar errores de integración antes de fusionar cambios.	Alta	2
HU-05	Como estudiante de otra tesis, quiero seguir una plantilla de contribución para poder conectar mi módulo al paquete sin romper la estructura existente.	Media	3
HU-06	Como evaluador del proyecto, quiero ejecutar una demo reproducible para poder verificar la integración de al menos datos, indicadores y un módulo analítico.	Alta	3

10. CRONOGRAMA DE ACTIVIDADES.

Actividad	Mes 1	Mes 2	Mes 3	Mes 4
Revisión y análisis de las referencias bibliográficas	X			
Levantamiento de información de los entregables que tendrán los subproductos de las otras tesis.	X			
Diseño de la estructura del paquete.	X			
Diseño de modelo Pydantic		X		
Desarrollo de endpoints principales de la API (FastAPI)		X		
Integración del módulo de datos y de indicadores		X	X	
Integración de un módulo analítico (IA)			X	
Pruebas unitarias y ajuste de cobertura mínima			X	
Documentación técnica, ejemplos de uso y plantilla de contribución				X
Demo técnica de integración y entrega de la versión v0.5				X

11. PRESUPUESTO Y FINANCIAMIENTO.

11.1. Presupuesto de componente técnico.

Rubro	Descripción	Costo (USD)
Laptop	Equipo en donde se desarrollará la librería y la API.	$1,210,00
Hosting de documentación	Publicación de documentación técnica en GitHub Pages	$10,00
Laptop	Equipo en donde se realizarán las pruebas unitarias.	$750,00
Licencia de Office	Licencia de Word en donde se realizará la documentación técnica.	$20,00
Conectividad.	Internet que se utilizará en el desarrollo de la librería y la API.	$200.00
TOTAL		**$2,190,00**

11.2. Presupuesto del recurso operativo.

Rubro	Descripción	Costo (USD)
Estudiante A.	Responsable de desarrollar el backend de la API: Estructura del paquete, endpoints, modelos Pydantic, servicios principales e instalación.	$5,000,00
Estudiante B.	Publicación de documentación técnica en GitHub Pages	$5,000,00
TOTAL		**$10,000.00**

11.3. Financiamiento.

11.3.1. Presupuesto de componente técnico.

Rubro	Descripción	Costo (USD)	Financiamiento.
Laptop	Equipo en donde se desarrollará la librería y la API.	$1,210,00	Financiamiento propio
Hosting de documentación	Publicación de documentación técnica en GitHub Pages	$10,00	Financiamiento propio
Laptop	Equipo en donde se realizarán las pruebas unitarias.	$750,00	Financiamiento propio
Licencia de Office	Licencia de Word en donde se realizará la documentación técnica.	$20,00	Financiamiento propio
Conectividad.	Internet que se utilizará en el desarrollo de la librería y la API.	$200.00	Financiamiento propio
TOTAL		**$2,190,00**	

11.3.2. Presupuesto del recurso operativo.

Rubro	Descripción	Costo (USD)	Financiamiento
Estudiante A.	Responsable de desarrollar el backend de la API: Estructura del paquete, endpoints, modelos Pydantic, servicios principales e instalación.	$5,000,00	Financiamiento propio

Estudiante B.	Publicación de documentación técnica en GitHub Pages	$5,000,00	Financiamiento propio
TOTAL		**$10,000.00**	

12. BENEFICIOS ESPERADOS DEL PROYECTO.

- Provee a las ocho tesis relacionadas con el gemelo digital multimodal una estructura común que reduce el riesgo de incompatibilidades en la etapa de integración final del FCI.
- Entrega un paquete Python reutilizable (georisk_twin) que puede servir de base para futuros proyectos de gemelos digitales orientados a riesgo urbano o industrial, dentro y fuera de la institución.
- Documenta de forma reproducible una arquitectura de integración basada en FastAPI y Pydantic, aportando un caso aplicado a la literatura existente sobre frameworks de gemelos digitales de código abierto.

13. DERECHOS DE PROPIEDAD INTELECTUAL.

El desarrollo de este trabajo de tesis contempla la generación de resultados susceptibles de protección por propiedad intelectual (derechos de autor, patentes, programa de computador).

¿El proyecto genera productos protegibles por propiedad intelectual?

- ☒ Software
- ☐ Modelo/algoritmo
- ☒ Manual/documentación técnica
- ☐ No aplica

Los derechos patrimoniales de los productos generados en el presente trabajo de tesis corresponderán a la institución, de conformidad con la normativa vigente. Los autores conservarán los derechos morales sobre su obra.

14. REFERENCIAS BIBLIOGRÁFICAS.

Ariyachandra, M. R. M. F., & Wedawatta, G. (2023). Digital twin smart cities for disaster risk management: A review of evolving concepts. Sustainability, 15(15), Artículo 11910. https://doi.org/10.3390/su151511910

Conde, J., Munoz-Arcentales, A., Alonso, Á., López-Pernas, S., & Salvachúa, J. (2023). Modeling digital twin data and architecture: A building guide with FIWARE as enabling technology. arXiv. https://doi.org/10.48550/arXiv.2309.12358

Gil Arboleda, S., Mikkelsen, P. H., Gomes, C., & Larsen, P. G. (2024). Survey on open-source digital twin frameworks–A case study approach. Software: Practice and Experience, 54(6), 929–960. https://doi.org/10.1002/spe.3305

Hevner, A. R., Parsons, J., Brendel, A. B., Lukyanenko, R., Tiefenbeck, V., Tremblay, M. C., & vom Brocke, J. (2024). Transparency in design science research. Decision Support Systems, 182, Artículo 114236. https://doi.org/10.1016/j.dss.2024.114236

Infante, S., Martín, C., Robles, J., Rubio, B., Díaz, M., González Perea, R., Montesinos, P., & Camacho Poyato, E. (2024). Integrating FMI and ML/AI models on the open-source digital twin framework OpenTwins. Software: Practice and Experience, 54(8), 1470–1490. https://doi.org/10.1002/spe.3322

Jeremiah, S. R., El Azzaoui, A., Xiong, N. N., & Park, J. H. (2024). A comprehensive survey of digital twins: Applications, technologies and security challenges. Journal of Systems Architecture, 151, Artículo 103120. https://doi.org/10.1016/j.sysarc.2024.103120

Lagap, U., & Ghaffarian, S. (2024). Digital post-disaster risk management twinning: A review and improved conceptual framework. International Journal of Disaster Risk Reduction, 110, Artículo 104629. https://doi.org/10.1016/j.ijdrr.2024.104629

Möller, N., Nurhas, I., Mattick, X., Geisler, S., & Pawlowski, J. (2022). Development of a method for the engineering of digital innovation using Design Science Research. Information, 13(12), Artículo 573. https://doi.org/10.3390/info13120573

Robles, J., Martín, C., & Díaz, M. (2023). OpenTwins: An open-source framework for the development of next-gen compositional digital twins. Computers in Industry, 152, Artículo 104007. https://doi.org/10.1016/j.compind.2023.104007

Rodríguez-Alonso, C., Pena-Regueiro, I., & García, Ó. (2024). Digital twin platform for water treatment plants using microservices architecture. Sensors, 24(5), Artículo 1568. https://doi.org/10.3390/s24051568

Tao, F., Sun, X., Cheng, J., Zhu, Y., Liu, W., Wang, Y., Xu, H., Hu, T., Liu, X., Liu, T., Sun, Z., Xu, J., Bao, J., Xiang, F., & Jin, X. (2024). makeTwin: A reference architecture for digital twin software platform. Chinese Journal of Aeronautics, 37(1), 1–18. https://doi.org/10.1016/j.cja.2023.05.002

Weil, C., Bibri, S. E., Longchamp, R., Golay, F., & Alahi, A. (2023). Urban digital twin challenges: A systematic review and perspectives for sustainable smart cities. Sustainable Cities and Society, 99, Artículo 104862. https://doi.org/10.1016/j.scs.2023.104862

Ozkan, N. (2022). Scrum, Kanban or a mix of both? A systematic literature review. En M. Ganzha, L. Maciaszek, M. Paprzycki, & D. Ślęzak (Eds.), Proceedings of the 17th Conference on Computer Science and Intelligence Systems (pp. 883–893). ACSIS. https://doi.org/10.15439/2022F143

Beaulieu, N., Dascalu, S. M., & Hand, E. (2022). API-first design: A survey of the state of academia and industry. En S. Latifi (Ed.), ITNG 2022: 19th International Conference on Information Technology-New Generations (Advances in Intelligent Systems and Computing, Vol. 1421, pp. 73–79). Springer. https://doi.org/10.1007/978-3-030-97652-1_10

Casas, S., Cruz, D., Vidal, G., & Constanzo, M. (2021). Uses and applications of the OpenAPI/Swagger specification: A systematic mapping of the literature. En 2021 40th

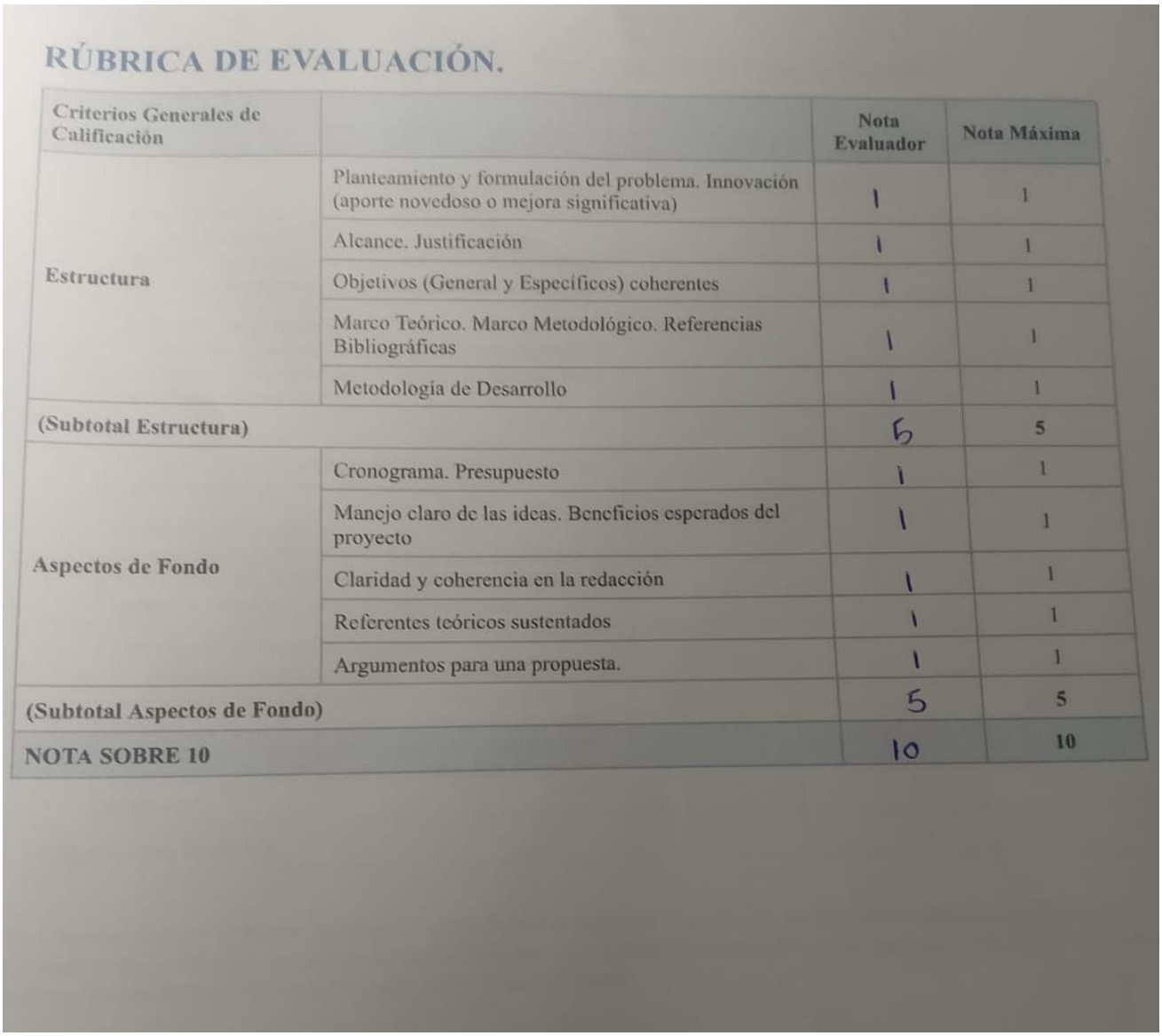
International Conference of the Chilean Computer Science Society (SCCC) (pp. 1–8).

IEEE. https://doi.org/10.1109/SCCC54552.2021.9650408

ANEXO — Ficha técnica del producto.

- **Producto:** Librería y API en el paquete georisk_twin.
- **Responsabilidad Estudiante A (Zambrano Mendoza Jeremy Dario):** Backend de API , estructura del paquete, endpoints, modelos Pydantic, servicios principales e instalación.
- **Responsabilidad Estudiante B (Chango Burgos Michael Alexander):** Integración y calidad , conexión de módulos de tesis, pruebas unitarias, documentación, ejemplos reproducibles y demo técnica.
- **Diseño experimental/alcance:** Prueba de integración horizontal de módulos; evaluación de instalación, endpoints, tiempo de respuesta y documentación.
- **Métodos y tecnologías:** Python, FastAPI, Pydantic, GeoPandas, NetworkX, scikit-learn, pytest, GitHub, documentación Markdown/OpenAPI.
- **Métricas clave:** Número de módulos integrados, cobertura mínima de pruebas, tiempo de respuesta, facilidad de instalación, reproducibilidad, documentación.
- **Dataset/Fuentes:** Subproductos de SI-01, SI-02, IND-SIS-01, URB-01, URB-02, URB-03, URB-04 y URB-05.
- **Criterios de aceptación:** Paquete instalable, API documentada, demo reproducible y conexión de al menos datos, indicadores y un módulo analítico.
- **Entregable verificable:** Paquete georisk_twin v0.5 + API documentada + demo de integración + guía técnica.
- **Recomendación metodológica:** Design Science Research + ingeniería de software + integración continua básica.
- **Dependencia/integración horizontal:** Tesis integradora horizontal; conecta todos los subproductos en el producto mínimo viable (MVP del FCI).
- **Riesgo principal y mitigación:** Dependencia de entregas de otros equipos; se mitiga con mocks, contratos de datos y plantillas de contribución.

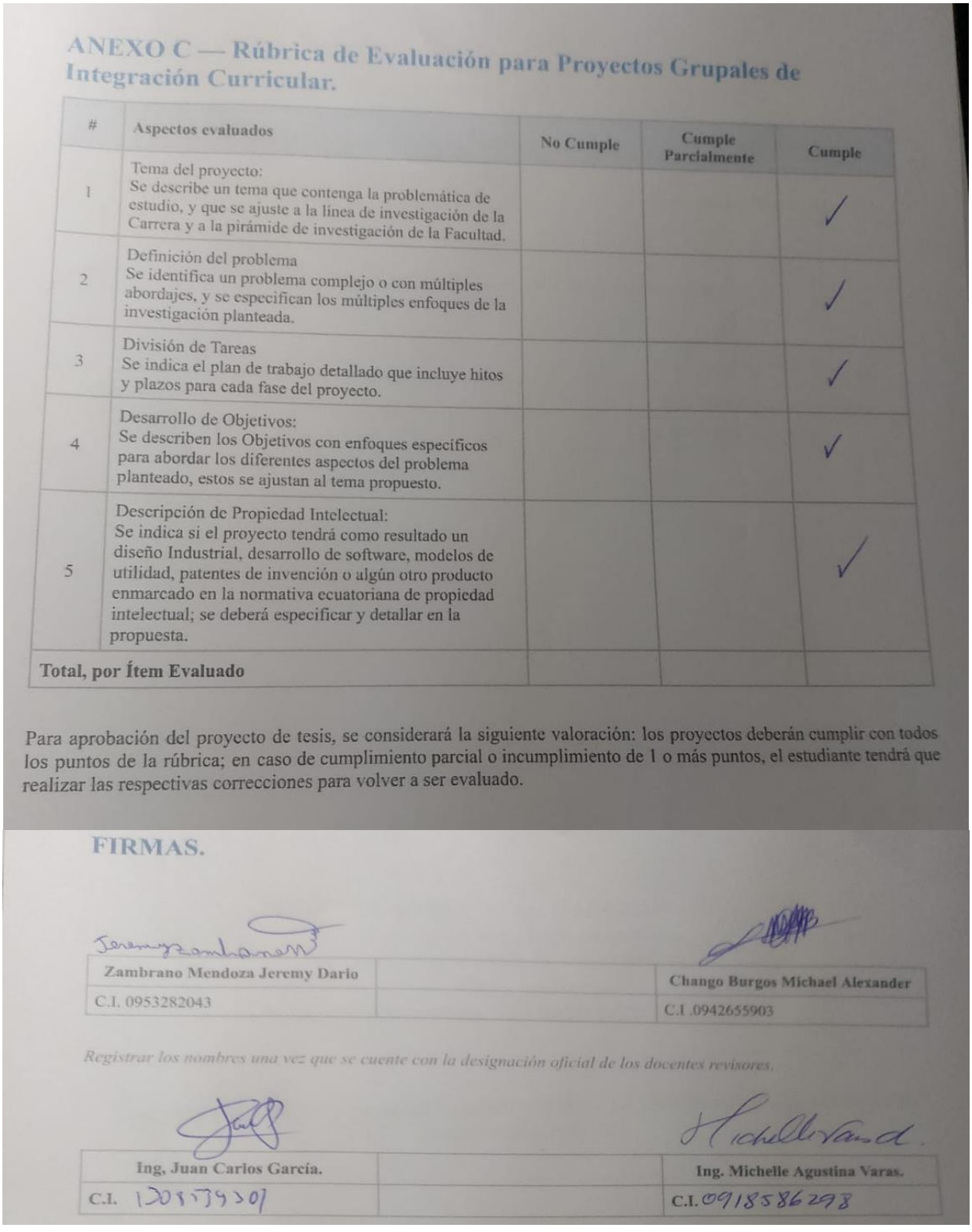
RÚBRICA DE EVALUACIÓN.



RÚBRICA DE EVALUACIÓN.

Criterios Generales de Calificación		Nota Evaluador	Nota Máxima
Estructura	Planteamiento y formulación del problema. Innovación (aporte novedoso o mejora significativa)	1	1
	Alcance. Justificación	1	1
	Objetivos (General y Específicos) coherentes	1	1
	Marco Teórico. Marco Metodológico. Referencias Bibliográficas	1	1
	Metodología de Desarrollo	1	1
(Subtotal Estructura)		5	5
Aspectos de Fondo	Cronograma. Presupuesto	1	1
	Manejo claro de las ideas. Beneficios esperados del proyecto	1	1
	Claridad y coherencia en la redacción	1	1
	Referentes teóricos sustentados	1	1
	Argumentos para una propuesta.	1	1
(Subtotal Aspectos de Fondo)		5	5
NOTA SOBRE 10		10	10

ANEXO C — Rúbrica de Evaluación para Proyectos Grupales de Integración Curricular.



ANEXO C — Rúbrica de Evaluación para Proyectos Grupales de Integración Curricular.

#	Aspectos evaluados	No Cumple	Cumple Parcialmente	Cumple
1	Tema del proyecto: Se describe un tema que contenga la problemática de estudio, y que se ajuste a la línea de investigación de la Carrera y a la pirámide de investigación de la Facultad.			✓
2	Definición del problema Se identifica un problema complejo o con múltiples abordajes, y se especifican los múltiples enfoques de la investigación planteada.			✓
3	División de Tareas Se indica el plan de trabajo detallado que incluye hitos y plazos para cada fase del proyecto.			✓
4	Desarrollo de Objetivos: Se describen los Objetivos con enfoques específicos para abordar los diferentes aspectos del problema planteado, estos se ajustan al tema propuesto.			✓
5	Descripción de Propiedad Intelectual: Se indica si el proyecto tendrá como resultado un diseño Industrial, desarrollo de software, modelos de utilidad, patentes de invención o algún otro producto enmarcado en la normativa ecuatoriana de propiedad intelectual; se deberá especificar y detallar en la propuesta.			✓
Total, por Ítem Evaluado				

Para aprobación del proyecto de tesis, se considerará la siguiente valoración: los proyectos deberán cumplir con todos los puntos de la rúbrica; en caso de cumplimiento parcial o incumplimiento de 1 o más puntos, el estudiante tendrá que realizar las respectivas correcciones para volver a ser evaluado.

FIRMAS.

Zambrano Mendoza Jeremy Dario		Chango Burgos Michael Alexander
C.I. 0953282043		C.I .0942655903

Registrar los nombres una vez que se cuente con la designación oficial de los docentes revisores.

Ing. Juan Carlos García.		Ing. Michelle Agustina Varas.
C.I. 1203739301		C.I. 0918586298

ANEXO_1_INTEGRACIÓN CURRICULAR.

UNIVERSIDAD DE GUAYAQUIL

LA MÁS GRANDE, IMPORTANTE Y GLORIOSA del Ecuador

ANEXOS

ANEXO I.- FORMATO DE EVALUACIÓN DE LA PROPUESTA DE TRABAJO DE INTEGRACIÓN CURRICULAR

FACULTAD DE INGENIERÍA INDUSTRIAL
CARRERA DE SISTEMAS DE INFORMACIÓN

Nombre de la propuesta de trabajo de integración curricular:	Librería / Api en python para operacionalizar la Metodología de un gemelo digital multimodal		
Nombre del estudiante (s):	Champo Burgos Richard Alexander Zambrano Mendoza Jeremy Dario		
Facultad:	Ingeniería Industrial	Carrera:	Sistemas de información
Línea de Investigación:	Ciencias Básicas, Diseño, Simulación y desarrollo Industrial	Sub-línea de Investigación:	Procesos Computacionales
Fecha de presentación de la propuesta de trabajo de integración curricular:	26/08/2026	Fecha de evaluación de la propuesta de trabajo de Integración curricular:	26/08/2026

ASPECTOS PARA CONSIDERAR	CUMPLIMIENTO SI	CUMPLIMIENTO NO	OBSERVACIONES
Título de la propuesta de trabajo de integración curricular:	✓		• Se van a implementar más metodología para el desarrollo del proyecto • Mejorar los objetivos específicos.
Línea de Investigación / Sublínea de Investigación:	✓		
Planteamiento del Problema:	✓		
Justificación e importancia:	✓		
Objetivos de la Investigación:	✓		
Metodología a emplearse:	✓		
Cronograma de actividades:	✓		
Presupuesto y financiamiento:	✓		

___ APROBADO
_✓_ APROBADO CON OBSERVACIONES
___ NO APROBADO

Ing. Manuel Samaniego Zamora, MBA
DECANO DE LA FACULTAD
CC: Director de Carrera, Gestor de Titulación

Facultad de Ingeniería Industrial
Av. Raúl Gómez Lince y Av. Juan Tanca Marengo
Guayaquil, Ecuador
ug.edu.ec